Prompt: Please code an example telnet server in Python that can reply to requests from a Java application.

Claude Sonnet 4.5: I'll create a simple telnet server in Python that can communicate with a Java client application.Now here's a Java client application that can connect to this server:I've created a Python telnet server and a Java client application that can communicate with each other.

Python Server Features:

- Listens on localhost:9000 (you can change the port)
- Handles multiple clients simultaneously using threads
- Responds to commands like "hello", "echo <message>", and "quit"
- Sends formatted responses back to clients

Java Client Features:

- Connects to the Python server
- Sends messages interactively
- Receives and displays responses
- Cleanly handles connection closure

To use them:

1. Run the Python server first:
   `python telnet_server.py`
2.
3. Compile and run the Java client:
   `javac TelnetClient.java`
4. `java TelnetClient`
5.
6. Try sending commands like:
   - `hello` - Gets a greeting
   - `echo test message` - Server echoes back
   - `quit` - Closes the connection

The server uses port 9000 instead of the standard telnet port 23 (which requires admin privileges). You can also test the server with a standard telnet client: `telnet localhost 9000`

Artifacts: telnet-server.py TelnetClient.java

Sample run:

```
% python3 telnet-server.py
Telnet server listening on localhost:9000
New connection from ('127.0.0.1', 62761)
Received from ('127.0.0.1', 62761): hello
Received from ('127.0.0.1', 62761): who are you?
Received from ('127.0.0.1', 62761): quit
Connection closed: ('127.0.0.1', 62761)

% java TelnetClient
Connected to localhost:9000
Welcome to Python Telnet Server
Enter message (or 'quit' to exit): hello
Server: Hello from Python server!
Enter message (or 'quit' to exit): who are you?
Server: You sent: who are you?
Enter message (or 'quit' to exit): quit
Server: Goodbye!
```

Test of multiple clients connecting to server:

```
% python3 telnet-server.py

New connection from ('127.0.0.1', 62762)
Received from ('127.0.0.1', 62762): I am java #1
Received from ('127.0.0.1', 62762): hello
New connection from ('127.0.0.1', 62763)
Received from ('127.0.0.1', 62763): I am Java #2
Received from ('127.0.0.1', 62763): hello
Received from ('127.0.0.1', 62763): quit
Connection closed: ('127.0.0.1', 62763)
Received from ('127.0.0.1', 62762): hello again from #1
Received from ('127.0.0.1', 62762): quit
Connection closed: ('127.0.0.1', 62762)

% java TelnetClient
Connected to localhost:9000
Welcome to Python Telnet Server
Enter message (or 'quit' to exit): I am java #1
Server: You sent: I am java #1
Enter message (or 'quit' to exit): hello
Server: Hello from Python server!
Enter message (or 'quit' to exit): hello again from #1
Server: You sent: hello again from #1
Enter message (or 'quit' to exit): quit
Server: Goodbye!

% java TelnetClient
Connected to localhost:9000
Welcome to Python Telnet Server
Enter message (or 'quit' to exit): I am Java #2
Server: You sent: I am Java #2
Enter message (or 'quit' to exit): hello
Server: Hello from Python server!
Enter message (or 'quit' to exit): quit
Server: Goodbye!
```

Here are the main ways to install a Java SDK on a Mac:

1. Using Homebrew (Recommended)

If you have Homebrew installed, this is the easiest method:

```
# Install the latest OpenJDK
brew install openjdk

# For a specific version (e.g., Java 17)
brew install openjdk@17

# Create a symlink so macOS can find it
sudo ln -sfn /opt/homebrew/opt/openjdk/libexec/
openjdk.jdk /Library/Java/JavaVirtualMachines/openjdk.jdk
```

If you don't have Homebrew, install it first from https://brew.sh

2. Download from Oracle or Adoptium

Oracle JDK:

- Visit https://www.oracle.com/java/technologies/downloads/
- Download the macOS installer (.dmg file)
- Double-click the .dmg and follow the installation wizard

Adoptium (Eclipse Temurin - Free & Open Source):

- Visit https://adoptium.net
- Select your macOS version and download the .pkg installer
- Run the installer

3. Using SDKMAN! (for managing multiple versions)

SDKMAN! is great if you need to switch between Java versions:

```
# Install SDKMAN!
curl -s "https://get.sdkman.io" | bash

# Install Java
sdk install java
```

Verify Installation

After installation, verify it worked:

```
java -version
javac -version
```

Set JAVA_HOME (if needed)

Add this to your `~/.zshrc` or `~/.bash_profile`:

```
export JAVA_HOME=$(/usr/libexec/java_home)
```

Which method would you prefer? Homebrew is usually the quickest if you already have it installed.

Here's how to update Java on a Raspberry Pi:

1. Update Using apt (Recommended)

If you installed Java through apt, simply update it:

```
# Update package list
sudo apt update

# Upgrade Java
sudo apt upgrade openjdk-17-jdk

# Or for the JRE only
sudo apt upgrade openjdk-17-jre
```

2. Install a Newer Version

To install a specific newer version:

```
# See available versions
apt search openjdk

# Install Java 17 (recommended for most projects)
sudo apt install openjdk-17-jdk

# Or Java 11
sudo apt install openjdk-11-jdk

# Or Java 21 (if available in your repos)
sudo apt install openjdk-21-jdk
```

3. Switch Between Java Versions

If you have multiple versions installed:

```
# Set default Java version
sudo update-alternatives --config java

# Set default Java compiler version
sudo update-alternatives --config javac
```

This will show you a list - enter the number for the version you want.

4. Check Current Version

```
java -version
javac -version
```

5. Using SDKMAN! (Alternative)

For more control over versions:

```
# Install SDKMAN!
curl -s "https://get.sdkman.io" | bash
source "$HOME/.sdkman/bin/sdkman-init.sh"

# Install Java
sdk install java 17.0.9-tem
```

Note on ARM Architecture

Make sure you're installing ARM-compatible versions. The Raspberry Pi uses ARM architecture, and OpenJDK from apt repositories is already optimized for it. Avoid downloading x86/x64 versions from Oracle's website.

What Java version are you currently running? You can check with `java -version` and I can help you decide if an update is needed.